



Lectura y escritura de **ARCHIVOS CSV** **EN PYTHON**

Melissa
Karen
Diego



Índice de CONTENIDOS

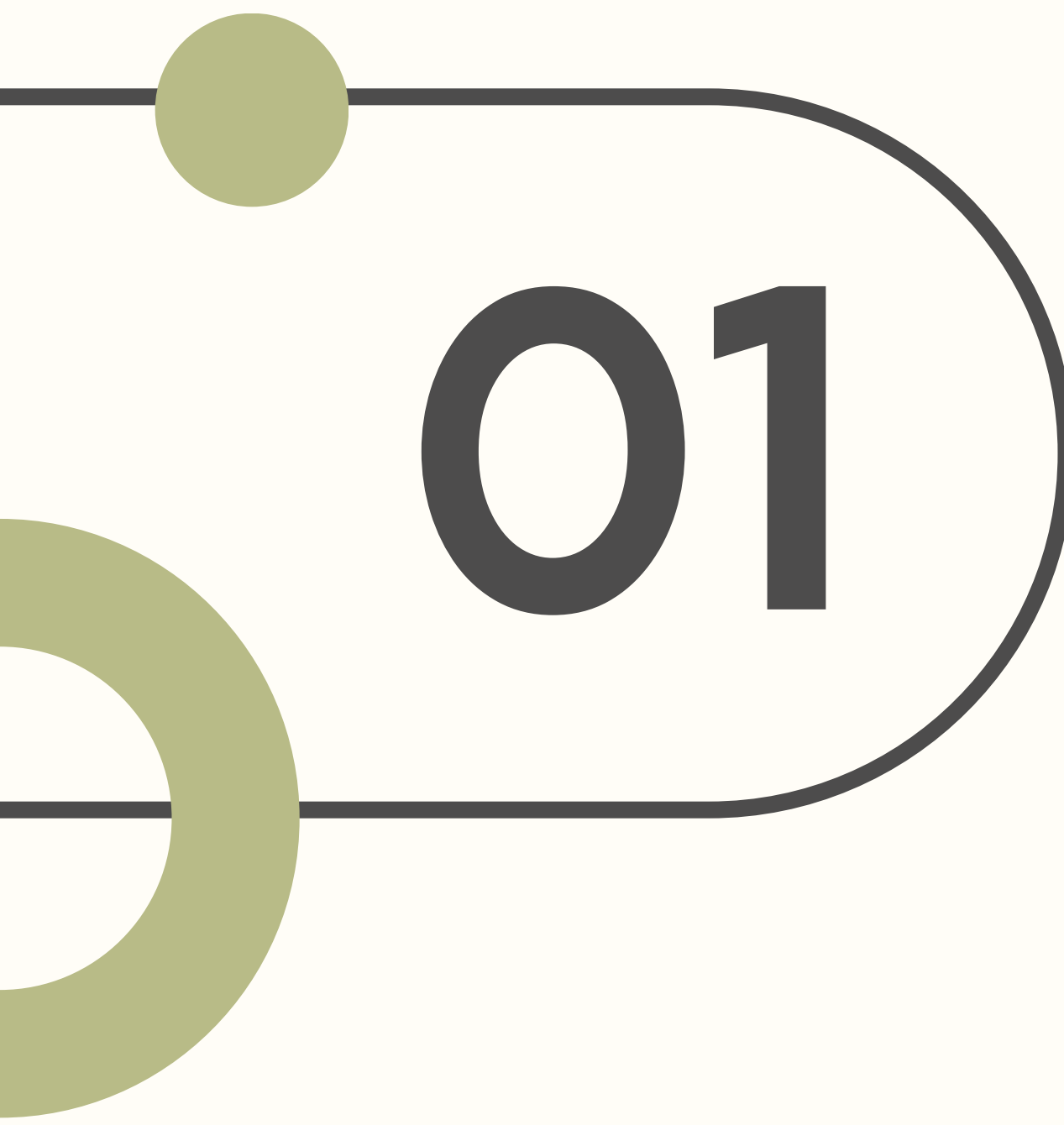


01. Qué es CSV

**02. Lectura y archivos CSV
con csv**

**03. Escribir archivos CSV
con csv**

**04. Analizar archivos CSV con
la pandas biblioteca**

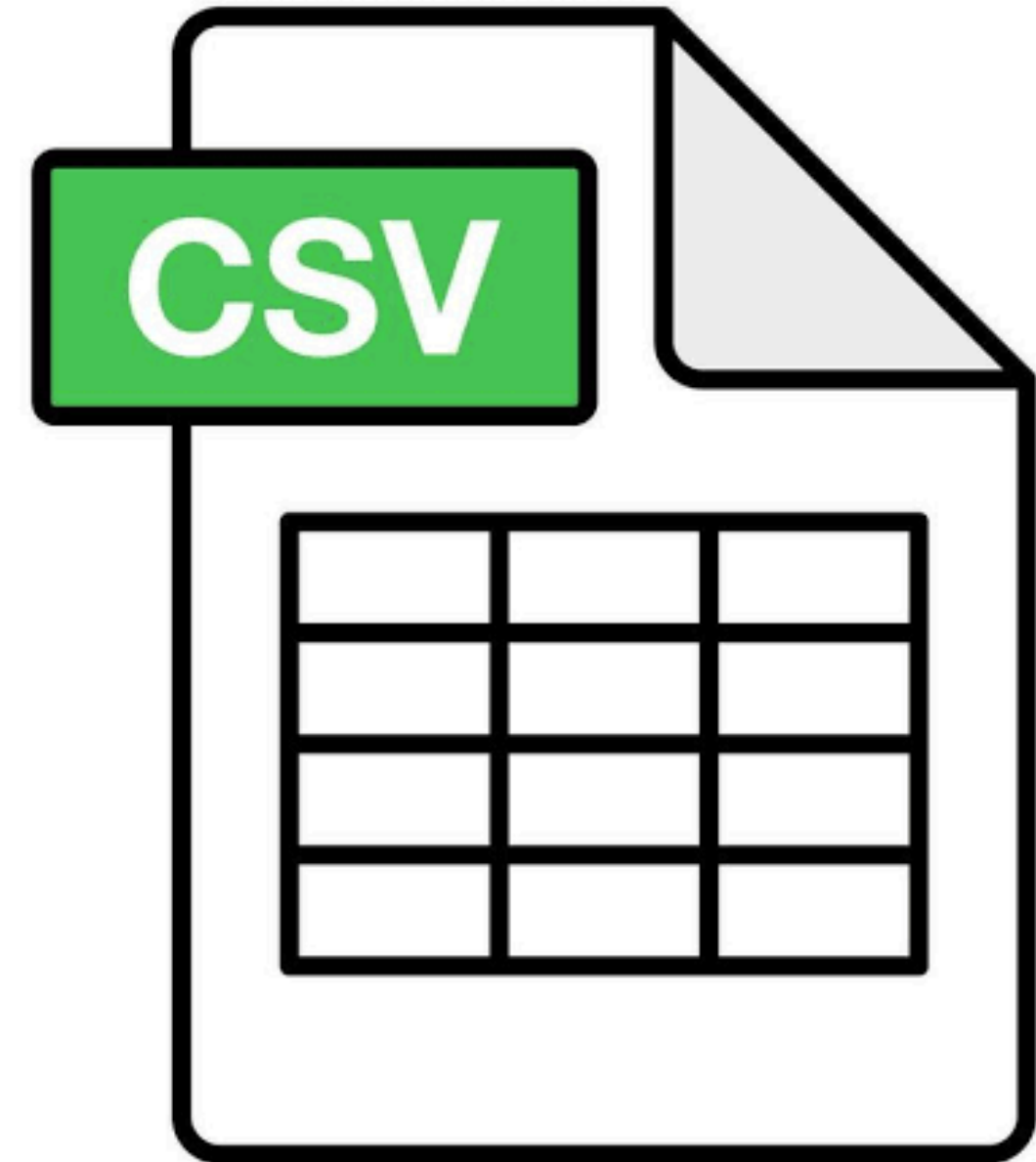


01

¿QUÉ ES CSV?

ACRÓNIMO: COMMA-SEPARATED VALUES (VALORES SEPARADOS POR COMAS)

Un archivo CSV es un archivo de texto que utiliza comas, o algún otro símbolo, para separar información. Es fácil leer y escribir en estos, como si fueran tablas.



CSV

```
column 1 name,column 2 name, column 3 name  
first row data 1,first row data 2,first row data 3  
second row data 1,second row data 2,second row data 3  
...
```

Cualquier lenguaje que admita la entrada de archivos de texto y la manipulación de cadenas (como Python) puede trabajar directamente con archivos CSV.

EJEMPLO

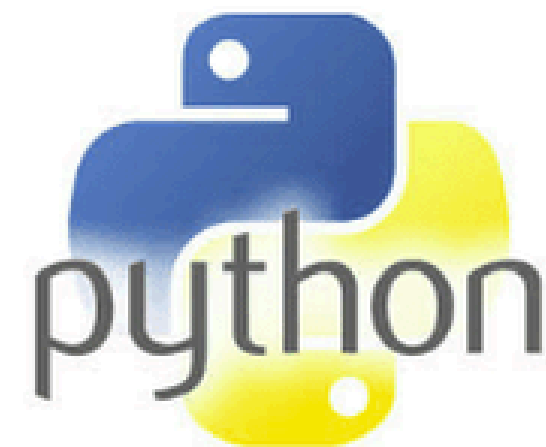
- Nombre,Edad,Color
Emilio,18,Rojo
Laura,21,Azul
Jimena,24,Rosa

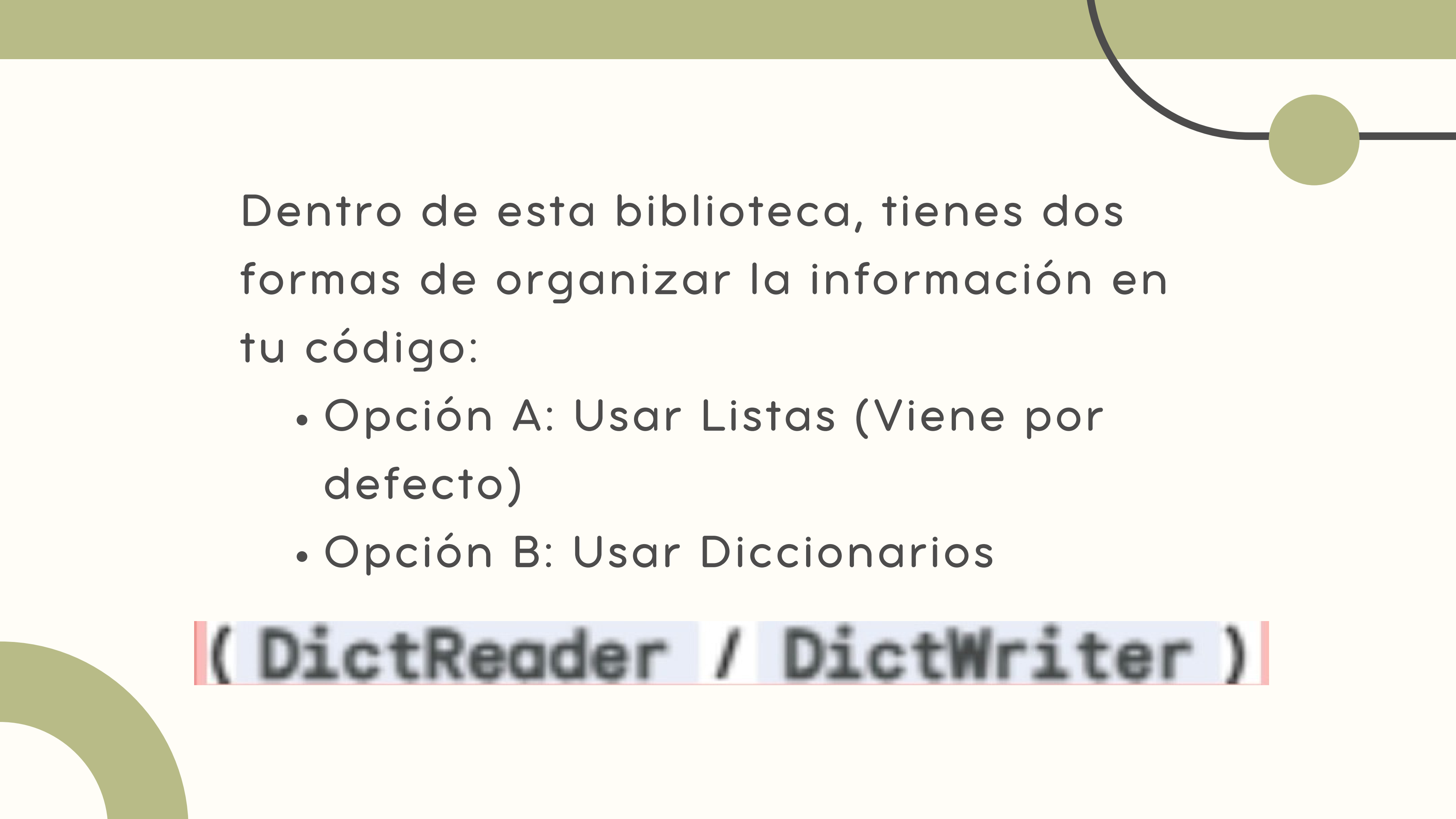
BIBLIOTECA CSV

csvbiblioteca permite leer y escribir archivos CSV.

Diseñada para funcionar directamente con archivos CSV generados por Excel

La csvbiblioteca incluye objetos y código para leer, escribir y procesar datos desde y hacia archivos CSV.

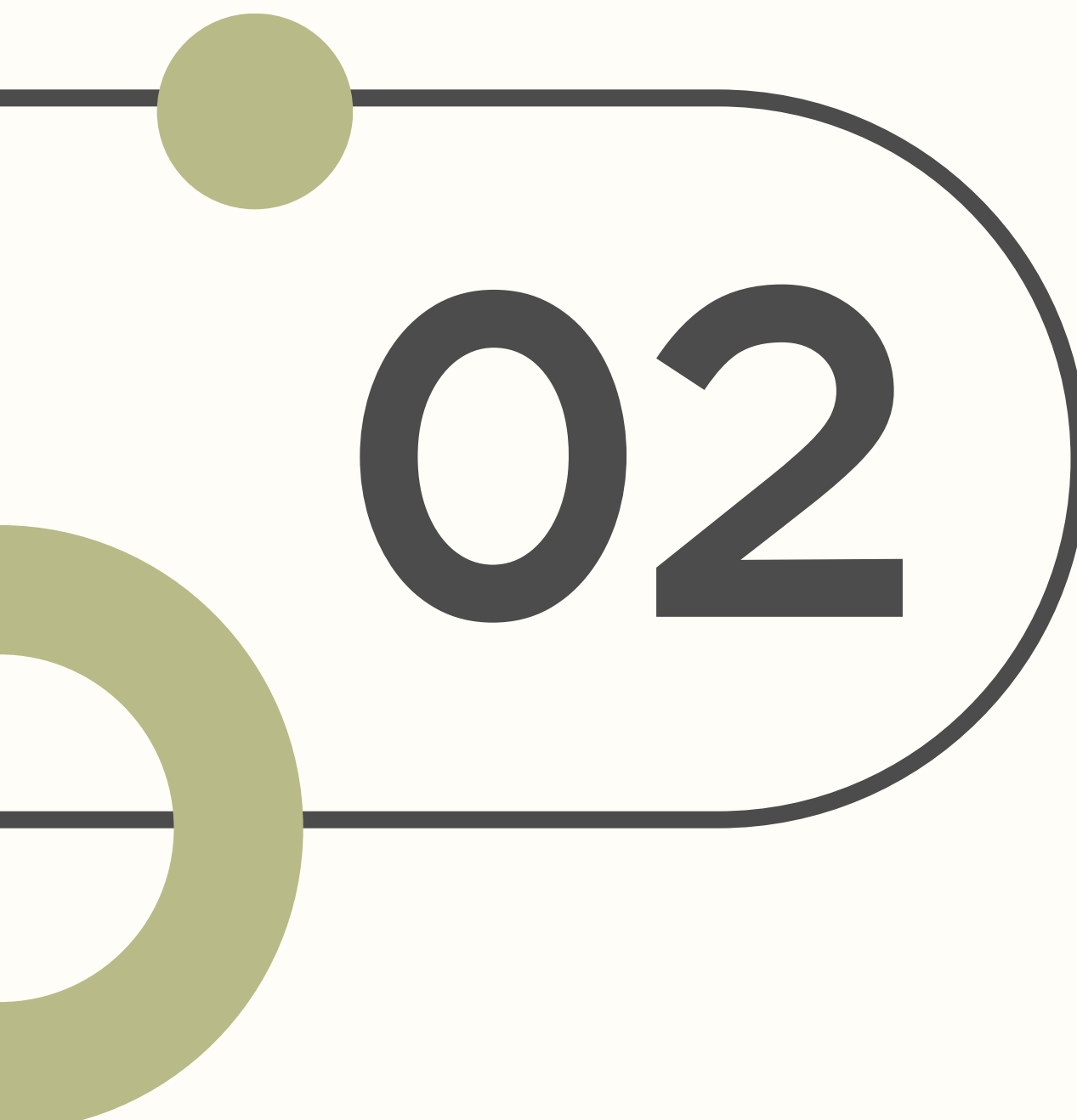




Dentro de esta biblioteca, tienes dos formas de organizar la información en tu código:

- Opción A: Usar Listas (Viene por defecto)
- Opción B: Usar Diccionarios

(DictReader / DictWriter)



02

LECTURA Y ARCHIVOS CSV CON CSV

OPCIÓN A

La lectura de un archivo CSV se realiza mediante el readerobjeto. Se abre como un archivo de texto con la función integrada de Python `open()`, que devuelve un objeto de archivo y se pasa luego a reader, que realiza el procesamiento principal.



1.

El archivo CSV se abre como un archivo de texto
`employee_birthday.csv`archivo:

CSV

```
name,department,birthday month
John Smith,Accounting,November
Erica Meyers,IT,March
```

2.

Código

```
Piton

import csv

with open('employee_birthday.csv') as csv_file:
    csv_reader = csv.reader(csv_file, delimiter=',')
    line_count = 0
    for row in csv_reader:
        if line_count == 0:
            print(f'Column names are {", ".join(row)}')
            line_count += 1
        else:
            print(f'\t{row[0]} works in the {row[1]} department, and was born in {row[2]}.')
            line_count += 1
    print(f'Processed {line_count} lines.')
```

3.

Esto produce el siguiente resultado:

Caparazón

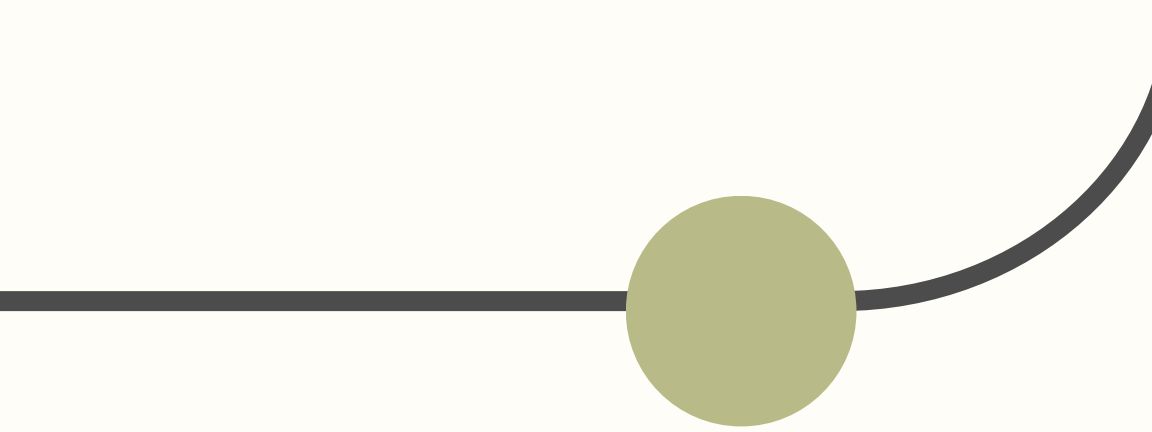
```
Column names are name, department, birthday month
  John Smith works in the Accounting department, and was born in November.
  Erica Meyers works in the IT department, and was born in March.
Processed 3 lines.
```


OPCIÓN B

(en un diccionario)

En lugar de trabajar con una lista de elementos individuales , también String puede leer los datos CSV directamente en un diccionario (técnicamente, un diccionario ordenado).





1.

El archivo CSV se abre como un archivo de texto
`employee_birthday.csv`archivo:

CSV

```
name,department,birthday month
John Smith,Accounting,November
Erica Meyers,IT,March
```



2.

Código

Pitón

```
import csv

with open('employee_birthday.csv', mode='r') as csv_file:
    csv_reader = csv.DictReader(csv_file)
    line_count = 0
    for row in csv_reader:
        if line_count == 0:
            print(f'Column names are {", ".join(row)}')
            line_count += 1
        print(f'\t{row["name"]} works in the {row["department"]} department, and was born in {row["birthday month"]}.' )
        line_count += 1
    print(f'Processed {line_count} lines.')
```


3.

Esto produce el mismo resultado:

Caparazón

```
Column names are name, department, birthday month
  John Smith works in the Accounting department, and was born in November.
  Erica Meyers works in the IT department, and was born in March.
Processed 3 lines.
```

DictReader

La primera línea del archivo CSV contiene las claves que se usarán para construir el diccionario. Si no las tiene, debe especificar sus propias claves configurando el `fieldnames` parámetro opcional con una lista que las contenga.

¿DE DÓNDE
PROVIENEN LAS
CLAVES DEL
DICCIONARIO?



READER PARÁMETROS OPCIONALES DE PYTHON PARA ARCHIVOS CSV

El readerobjeto puede
→ manejar diferentes estilos de
archivos CSV especificando
parámetros adicionales:



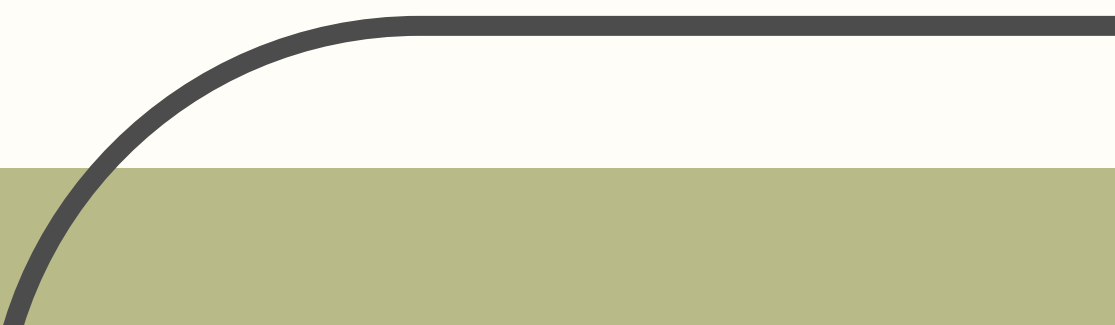


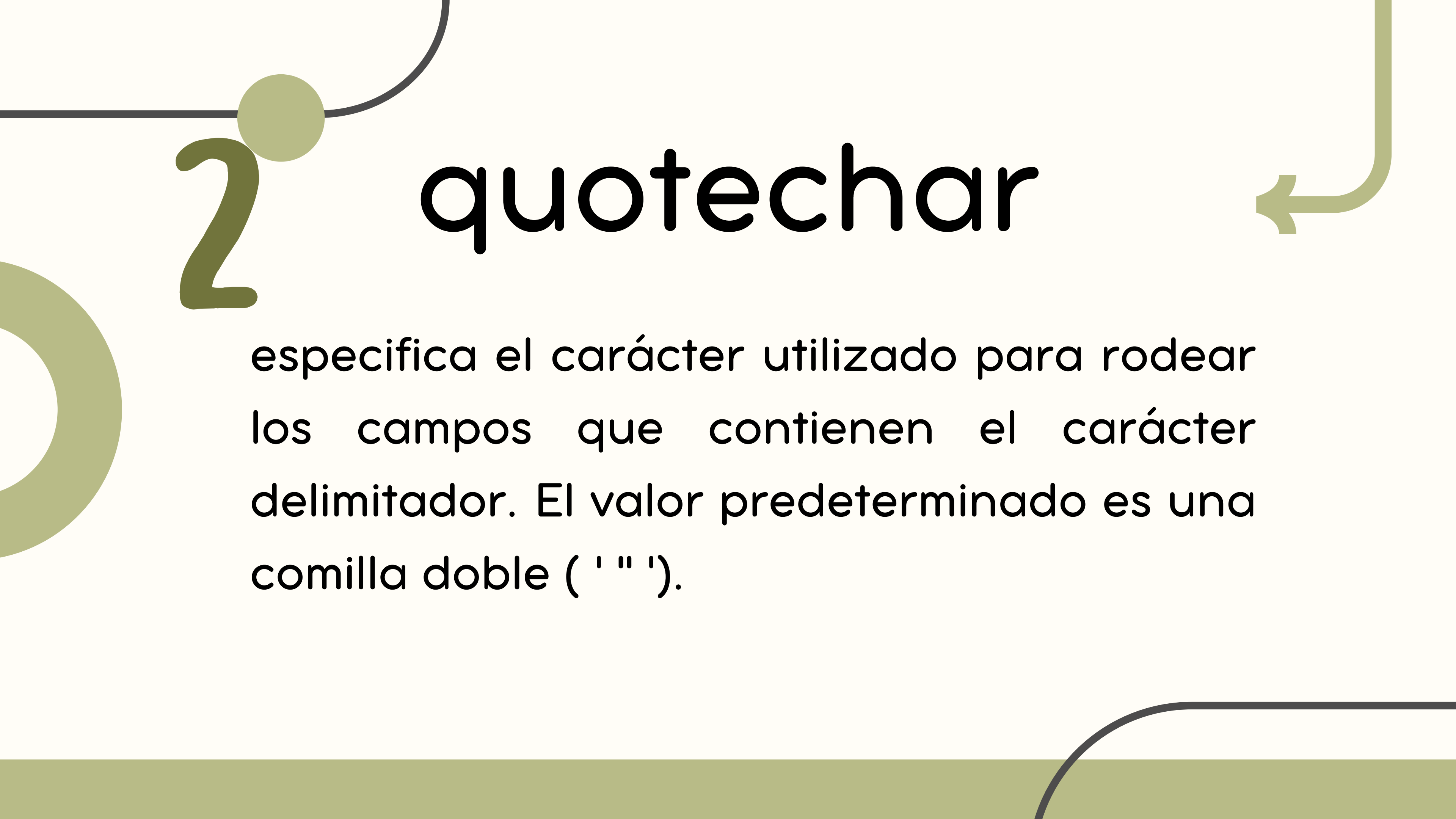
1.

delimiter



especifica el carácter utilizado para separar cada campo. El valor predeterminado es la coma (',').





2

quotechar

especifica el carácter utilizado para rodear los campos que contienen el carácter delimitador. El valor predeterminado es una comilla doble (' " ').

3. escapechar

Especifica el carácter que se utiliza para escapar el carácter delimitador, en caso de que no se utilicen comillas. Por defecto, no se utiliza ningún carácter de escape.

EJEMPLO

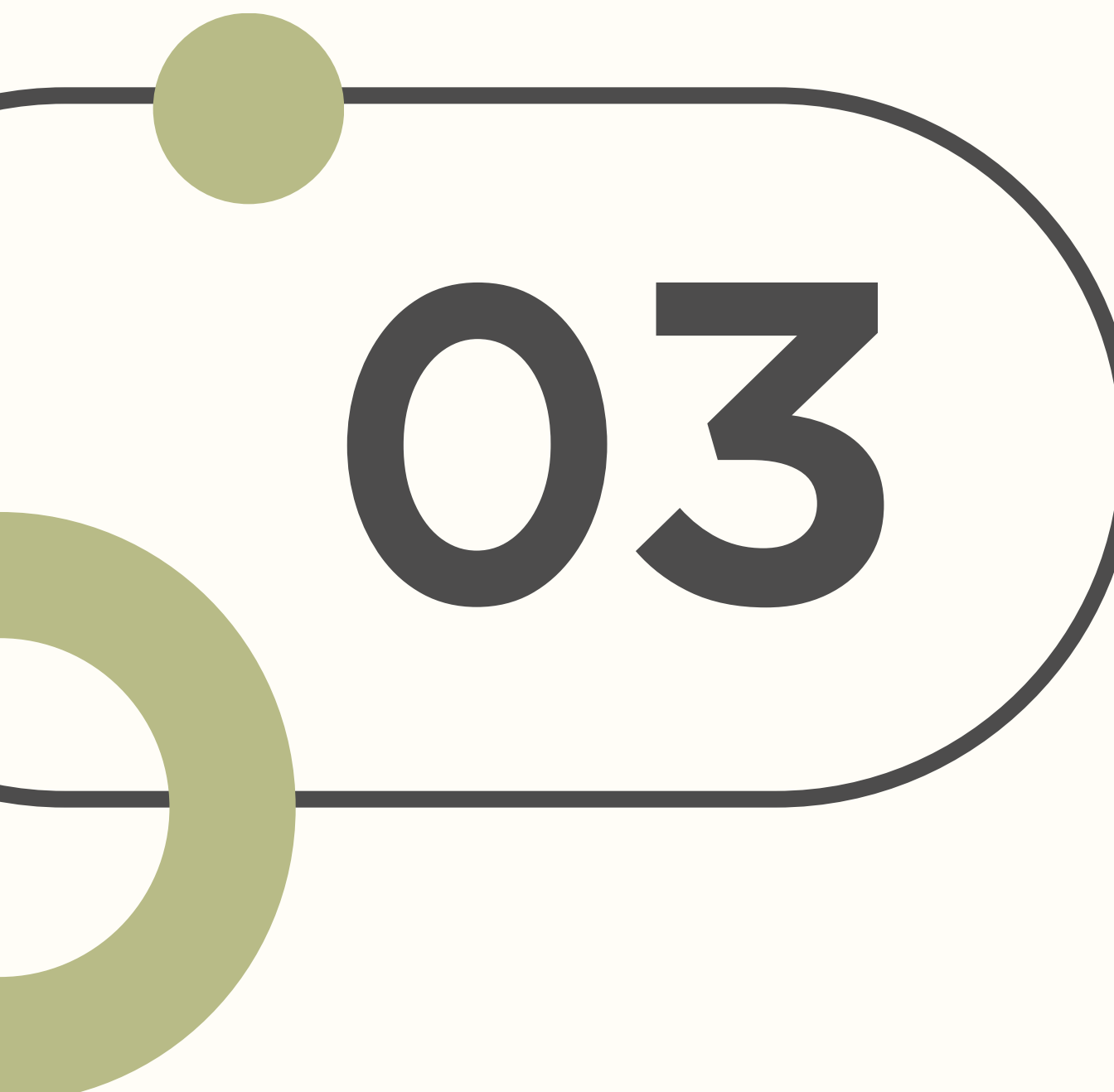
Problema: Este archivo CSV contiene tres campos: name, address, y date joined, que están separados por comas. El problema es que los datos del address campo también contienen una coma para indicar el código postal.

Solución:

1. Utilice un delimitador diferente
2. Encierra los datos entre comillas.
3. El "escudo"

CSV

```
name,address,date joined
john smith,1132 Anywhere Lane Hoboken NJ, 07030,Jan 4
erica meyers,1234 Smith Lane Hoboken NJ, 07030,March 2
```

03

ESCRIBIR

ARCHIVOS CSV

CON CSV

OPCIÓN A

También puedes escribir en un archivo CSV usando un writerobjeto y el `.write_row()` método:



código

Pitón

```
import csv

with open('employee_file.csv', mode='w') as employee_file:
    employee_writer = csv.writer(employee_file, delimiter=',', quotechar='"', quoting=csv.QUOTE_MINIMAL)

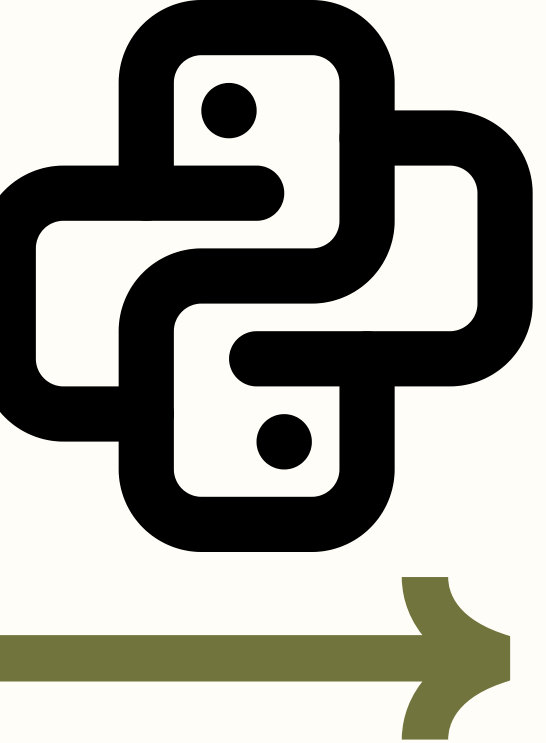
    employee_writer.writerow(['John Smith', 'Accounting', 'November'])
    employee_writer.writerow(['Erica Meyers', 'IT', 'March'])
```

resultado

CSV

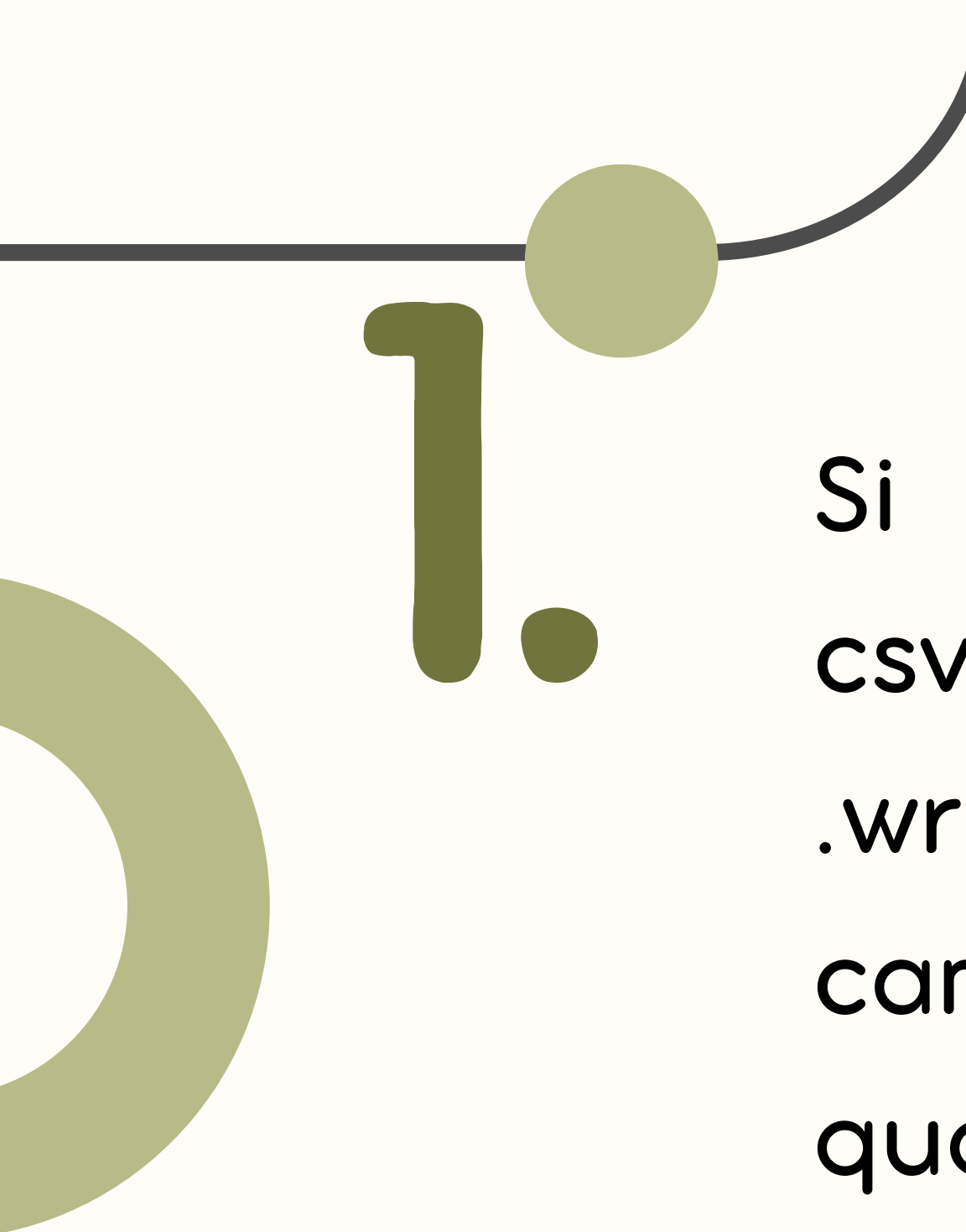
```
John Smith,Accounting,November
Erica Meyers,IT,March
```

¿A QUÉ SE REFIERE
QUOTING=CSV.QUOTE_MINIMAL?



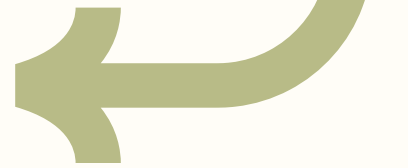
El `quotechar` parámetro opcional indica writer qué carácter usar para entrecomillar los campos al escribir. El uso o no de las comillas viene determinado por el `quoting` parámetro opcional

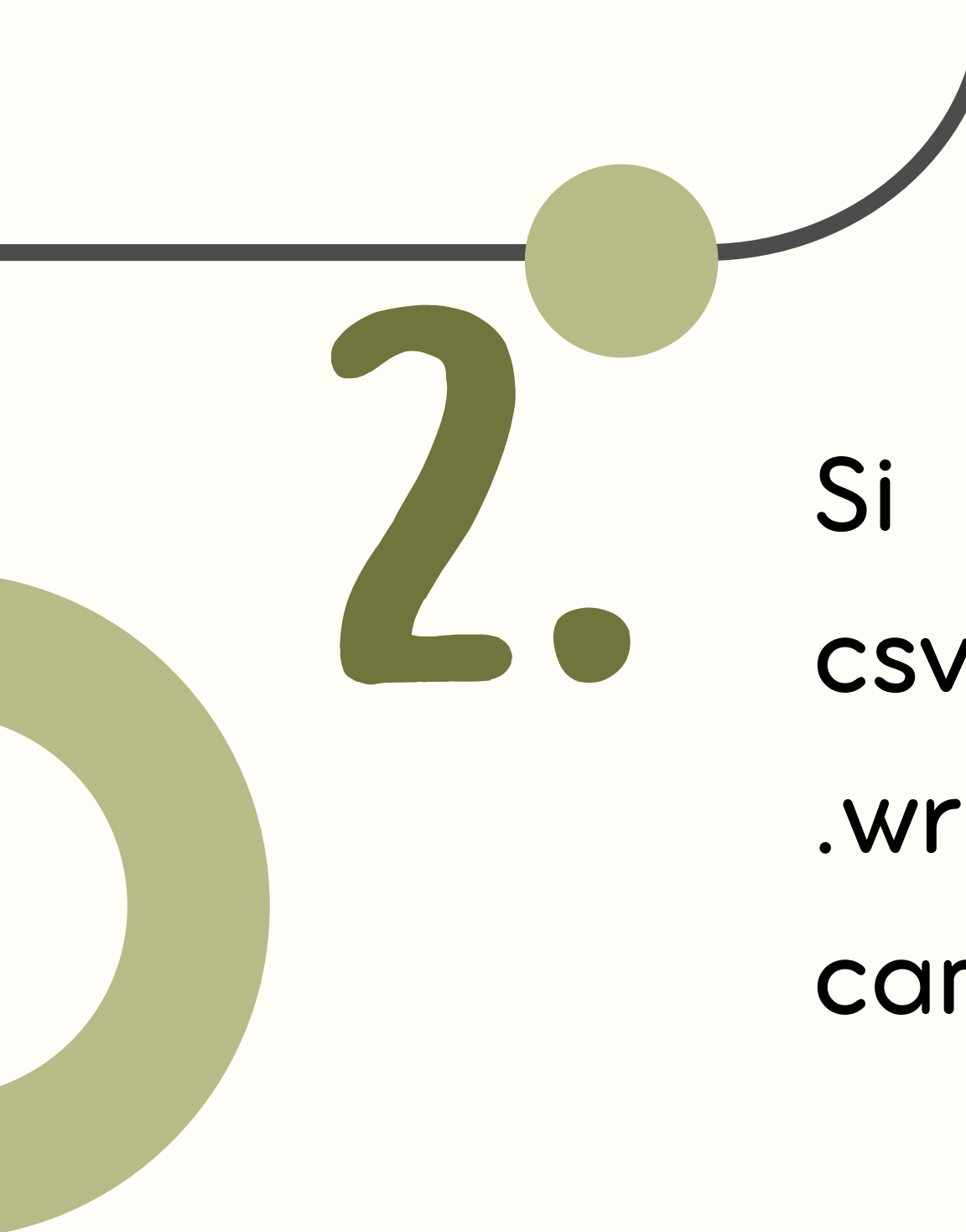




1.

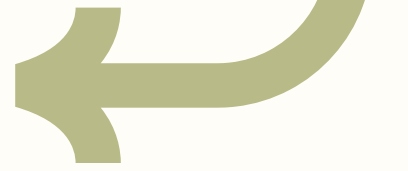
Si `quoting` se establece en `csv.QUOTE_MINIMAL`, entonces `.writerow()` solo se citarán los campos si contienen delimitero `quotechar`. Este es el caso predeterminado.

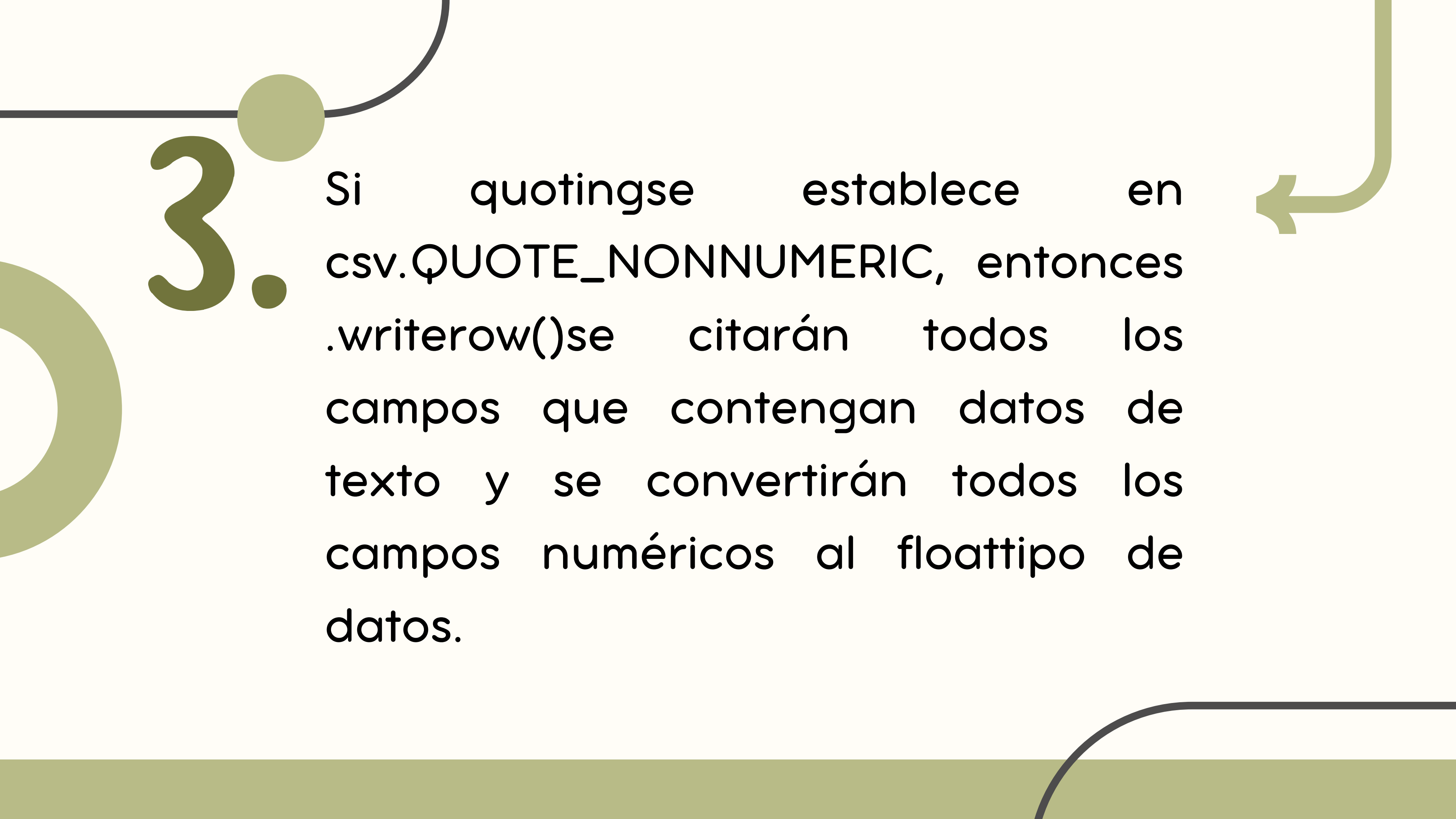




2.

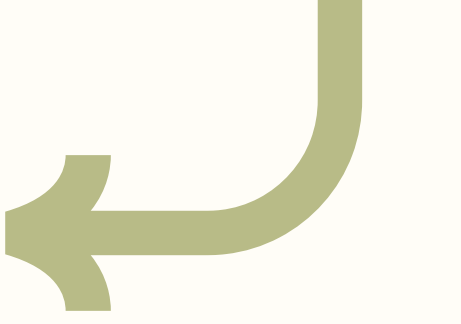
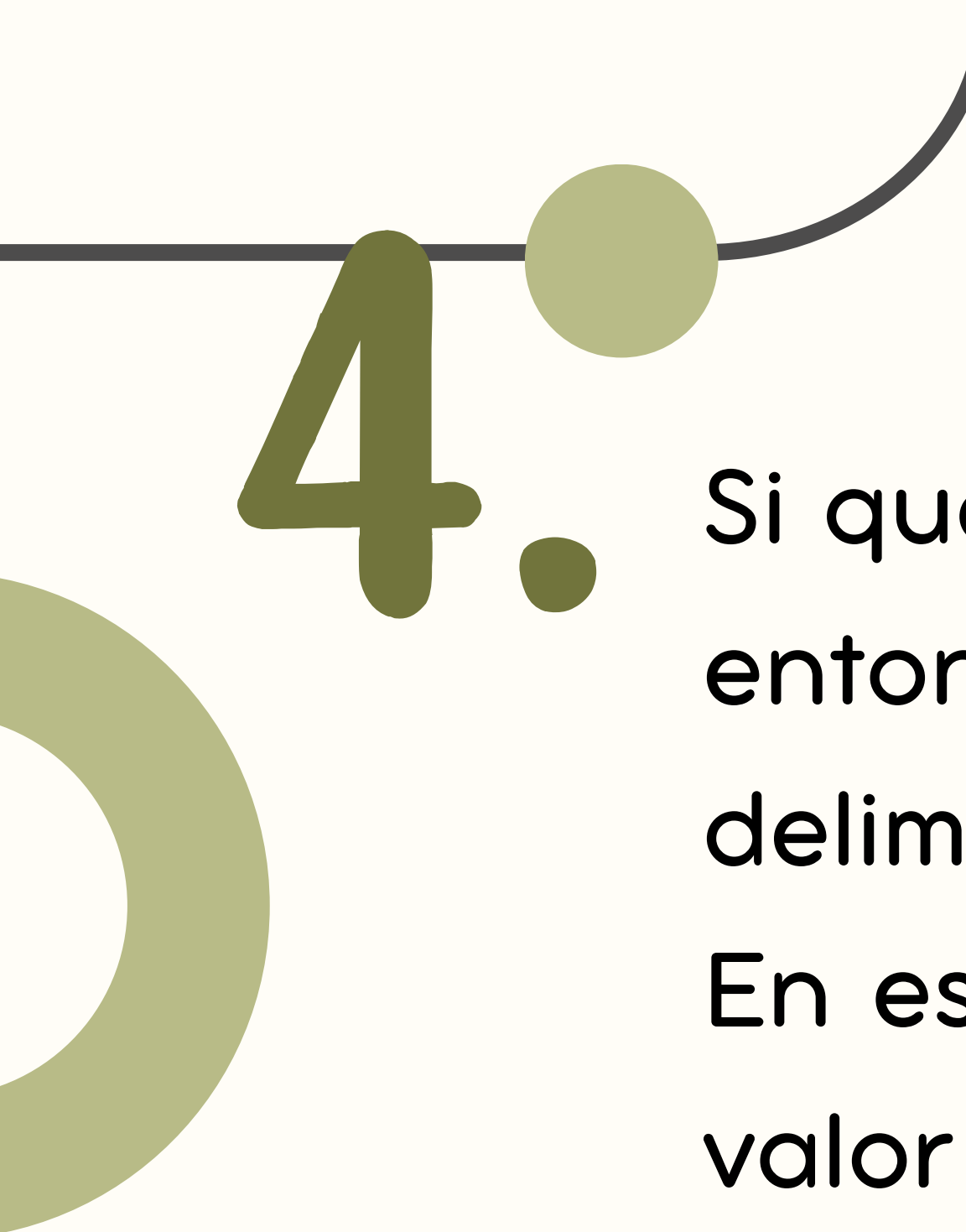
Si `quotingse` establece en `csv.QUOTE_ALL`, entonces `.writerow()` se citarán todos los campos





3.

Si `quotingse` establece en `csv.QUOTE_NONNUMERIC`, entonces `.writerow()` se citarán todos los campos que contengan datos de texto y se convertirán todos los campos numéricos al float tipo de datos.



4.

Si quoting se establece en `csv.QUOTE_NONE`, entonces `.writerow()` escapará los delimitadores en lugar de entrecomillarlos. En este caso, también debe proporcionar un valor para el escapechar parámetro opcional.

OPCIÓN B

(en un diccionario)

A diferencia de DictReader, el `fieldnames` parámetro es necesario al escribir un diccionario.. También usa las claves en `fieldnames` para escribir la primera fila como nombres de columna.

CSV module

Opening the CSV file

```
with open('C:/my_data/Product_Info.CSV', mode='r') as file:
```

importing the CSV file

```
df_CSV = CSV.reader(file)
```

displaying the contents of the CSV file

```
for lines in df_CSV:
```

```
    print(lines)
```

OUTPUT

```
Product ID', 'Product Name', 'Cost Price', 'Selling Price']  
1, 'Shirt', '$23', '$34']  
2, 'Bag', '$56', '$70']  
3, 'Socks', '$32', '$43']
```

código

Pitón

```
import csv

with open('employee_file2.csv', mode='w') as csv_file:
    fieldnames = ['emp_name', 'dept', 'birth_month']
    writer = csv.DictWriter(csv_file, fieldnames=fieldnames)

    writer.writeheader()
    writer.writerow({'emp_name': 'John Smith', 'dept': 'Accounting', 'birth_month': 'November'})
    writer.writerow({'emp_name': 'Erica Meyers', 'dept': 'IT', 'birth_month': 'March'})
```

resultado

CSV

```
emp_name,dept,birth_month
John Smith,Accounting,November
Erica Meyers,IT,March
```



TURNO DE...

PREGUNTAS (fáciles)

COMENTARIOS (bonitos)

CRÍTICAS (constructivas)

APORTACIONES (monetarias)

BIBLIOGRAFÍA

csv – CSV File Reading and Writing. (s. f.). Python Documentation.
<https://docs.python.org/es/3/library/csv.html>

Fincher, J. (2023, 25 enero). Reading and Writing CSV Files in Python.
<https://realpython.com/python-csv/>

csv – Lectura y escritura de archivos CSV – documentación de Python – 3.9.25. (s. f.). <https://docs.python.org/es/3.9/library/csv.html>



Muchas
GRACIAS

www.unsitiogenial.es

